

Universidade Federal do Rio Grande do Sul (UFRGS)
Instituto de Matemática e Estatística
Curso de Extensão: Python e matemática de dados financeiros:
conceitos e aplicações

Material Complementar
Tuplas: Contextualização e Aplicabilidade

Elaborado por Laura Kowalczyk

Porto Alegre
Abril de 2026

Contextualização

A partir de estruturas de programação, representamos informações do mundo real dentro de um código. No caso das tuplas, suas aplicabilidades são direcionadas à representação de conjuntos de dados estáticos.

Isto é, as tuplas são um tipo de estrutura de dados nativa do Python que pertencem ao grupo das sequências ordenadas. E uma característica essencial desse comando é sua imutabilidade, ou seja, depois de criada ela não pode ser modificada.

Estrutura

- **Estrutura**

A criação de uma tupla consiste em agrupar valores em uma única variável. A estrutura primordial para uma tupla são as vírgulas e, comumente, parênteses também podem ser vistos pois esses facilitam a leitura e interpretação do código.

Aqui temos duas formas de criar uma tupla de dois valores relacionados, com ou sem o parênteses:

```
coordenada = 10, 20
```

```
coordenada = (10, 20)
```

Até mesmo tuplas de um único elemento necessitam da presença de vírgulas, pois elas garantirão a identificação de que se trata, de fato, de uma tupla. Veja:

```
elemento_unico = (10,)
```

- **Acesso**

Os elementos de uma tupla podem ser acessados por meio de índices, começando do zero (assim como nas listas).

Isso permite recuperar valores específicos dentro da estrutura.

```
coordenada = (10, 20)
print(coordenada[0])
```

- **Imutabilidade**

Uma vez criada, a tupla não pode ser modificada. Isso significa que não é possível alterar, adicionar ou remover elementos. Essa característica garante maior segurança e previsibilidade dos dados.

```
coordenada[0] = 15 # erro
```

- **Desempacotamento**

O desempacotamento permite atribuir os valores da tupla diretamente a variáveis. Cada variável recebe um valor correspondente da tupla.

```
x, y = (10, 20)
```

- **Retorno de funções**

Funções em Python podem retornar múltiplos valores. Internamente, esses valores são agrupados em uma tupla.

Isso facilita o retorno de mais de um resultado de forma organizada.

```
def calcular(a, b):
    return a + b, a * b
```

Aplicabilidade

- **Tuplas, Listas, Sets e Dicts, quando utilizar cada um?**

- **Tuplas**

As tuplas, como já foi mencionado, são estruturas muito comuns quando queremos maior rigidez nas informações, dado que são imutáveis e armazenam dados que se manterão fixos.

Nesse sentido, podem ser vistas como mais seguras também, dado que não correm o risco de gerar alterações acidentais e também podem ser utilizadas como chaves de dicionários.

De forma mais prática, tuplas representam uma entidade única do múltiplos atributos.

- **Listas**

Em contraponto, temos as listas que são uma alternativa similar às tuplas. Essas, por sua vez, representam coleções dinâmicas de dados e permitem alterações.

Nesse sentido, podemos categorizá-las como mais flexíveis e que representam conjuntos de elementos que podem variar ao longo do tempo.

- **Sets**

Outra estrutura semelhante, mas com outras particularidades e finalidades são os Sets, que funcionam de forma muito semelhante aos conjuntos matemáticos. Essa semelhança se dá por serem ideais para verificações de permanência, isto é, está ou não dentro do set de elementos ou para intersecção, união ou diferenças entre eles

Suas distinções principais às tuplas são:

- não permitem elementos duplicados;
- não possuem uma ordenação fixa;
- permitem mudança de elementos.

- **Dicts**

Os dicts ou traduzindo, dicionários já são o formato que mais se distanciam dos tratados até aqui. Esses irão alterar a lógica de armazenamento dos dados, pois enquanto tuplas, listas e sets guardam valores isolados, os dicionários trabalham com pares de “chave-valor”. Em vez de acessar um dado por uma posição numérica (índice 0, 1, 2), a informação é acessada por meio de uma palavra-chave (uma etiqueta identificadora única).

Nas tuplas, o significado de um dado depende estritamente da sua posição (ex: sabemos que o índice 0 é o nome e o 1 é a idade apenas por convenção). No dicionário, essa associação é explícita e mutável, onde a própria chave diz o que o valor significa {'nome': 'Laura'}. Nesse sentido, podemos categorizá-los como estruturas ideais para mapeamento de dados complexos, funcionando como pequenos bancos de dados indexados muito rápidos.

Dadas essas diferenças e semelhanças, podemos dizer que dicts são recomendáveis quando é preciso associar propriedades textuais ou identificadores a determinados valores e necessita buscar essas informações de forma instantânea através de um nome, código ou ID.

- **Exemplos**

- **Exemplo 1:** Coordenadas geográficas

Um ponto no mapa tem latitude e longitude fixas.

Representação ideal: tupla

```
localizacao = (-30.0346, -51.2177)
```

- **Exemplo 2:** Cadastro de pessoa

Nome, idade e estado formam um conjunto único.

Representação ideal: tupla

```
pessoa = ("Laura", 25, "RS")
```

- **Exemplo 3:** Lista de tarefas

Tarefas podem ser adicionadas ou removidas.

Representação ideal: lista

```
tarefas = ["estudar", "treinar"]
tarefas.append("ler")
```

- **Exemplo 4:** Filtro de ativos operados

Ativos podem se repetir em um histórico de negociação, mas precisamos extrair apenas os tickers únicos operados no dia.

Representação ideal: set

```
# O set elimina automaticamente as duplicatas da lista de transações
ativos_negociados = {"PETR4", "VALE3", "PETR4", "ITUB4", "VALE3"}
print(ativos_negociados) # Saída: {'VALE3', 'PETR4', 'ITUB4'}
```

- **Exemplo 5:** Painel de cotações

Preços de ações mudam rapidamente e precisam ser consultados diretamente pelo nome do ativo, sem depender de índices numéricos.

Representação ideal: dicts

```
# Cada ticker (chave) aponta diretamente para o seu preço atual (valor)
quadro_cotacoes = {"PETR4": 36.50, "VALE3": 62.10}
print(quadro_cotacoes["PETR4"]) # Saída: 36.50
```

- **Outras Aplicações**

- **Indexação**

Anteriormente falamos sobre cada elemento de uma tupla possui uma posição indexada que começa no zero (0). No entanto, podemos trabalhar um pouco mais sobre esse tema, veja:

- **Índices Negativos (t[-1]):** Permitem acessar os elementos de trás para frente. Isto é, o índice -1 sempre representará o último elemento, o -2 o penúltimo, e assim por diante. Essa funcionalidade é extremamente útil quando não sabemos o tamanho exato da tupla, mas precisamos do último registro.
- **Fatiamentos ou Slicings (t[1:4]):** Permite extrair uma "subtupla", ou seja, um "pedaço" da tupla original. A sintaxe funciona como [início:fim], onde o índice de início é incluso e o índice de fim não é excluído do resultado.

Abaixo, a aplicabilidade conjunta das funcionalidades comentadas:

```
# Uma tupla representando o histórico de fechamento de uma ação em 5 dias
historico_precos = (34.50, 35.20, 36.00, 35.80, 36.50)

# t[1] -> Acessa o segundo elemento (índice 1)
print(historico_precos[1]) # Saída: 35.20

# t[-1] -> Acessa o último elemento da sequência
print(historico_precos[-1]) # Saída: 36.50

# t[1:4] -> Extrai do índice 1 até o índice 3 (o 4 fica de fora)
print(historico_precos[1:4]) # Saída: (35.20, 36.00, 35.80)
```

- Concatenação

Embora as tuplas sejam imutáveis após criação, o Python permite que elas sejam combinadas entre si a fim de uma nova tupla utilizando o operador de adição (+).

Lembrando que, essa operação de concatenação não altera nenhuma das tuplas originais na memória. Na realidade, ela cria uma terceira estrutura unindo os dados das envolvidas. Verifique:

```
# Cenário: Unir os dados de cadastro de dois períodos diferentes
trimestre_1 = ("PETR4", "VALE3")
trimestre_2 = ("ITUB4", "BBAS3")

# Concatenando as duas estruturas
carteira_semestral = trimestre_1 + trimestre_2

print(carteira_semestral)
# Saída: ('PETR4', 'VALE3', 'ITUB4', 'BBAS3')
```

Exercícios para Praticar

- **Exercício 1**

Represente uma coordenada (latitude, longitude) usando uma tupla e imprima cada valor separadamente.

Resolução

```
coordenada = (-30.0, -51.0)
lat, long = coordenada
print(lat)
print(long)
```

- **Exercício 2**

Uma função deve receber duas notas. A partir disso, definir a média do aluno e a situação (aprovado ou reprovado).

Resolução

```
def avaliar(n1, n2):  
    media = (n1 + n2) / 2  
  
    if media >= 7:  
        return media, "aprovado"  
    else:  
        return media, "reprovado"  
  
resultado = avaliar(8, 6)  
print(resultado)
```